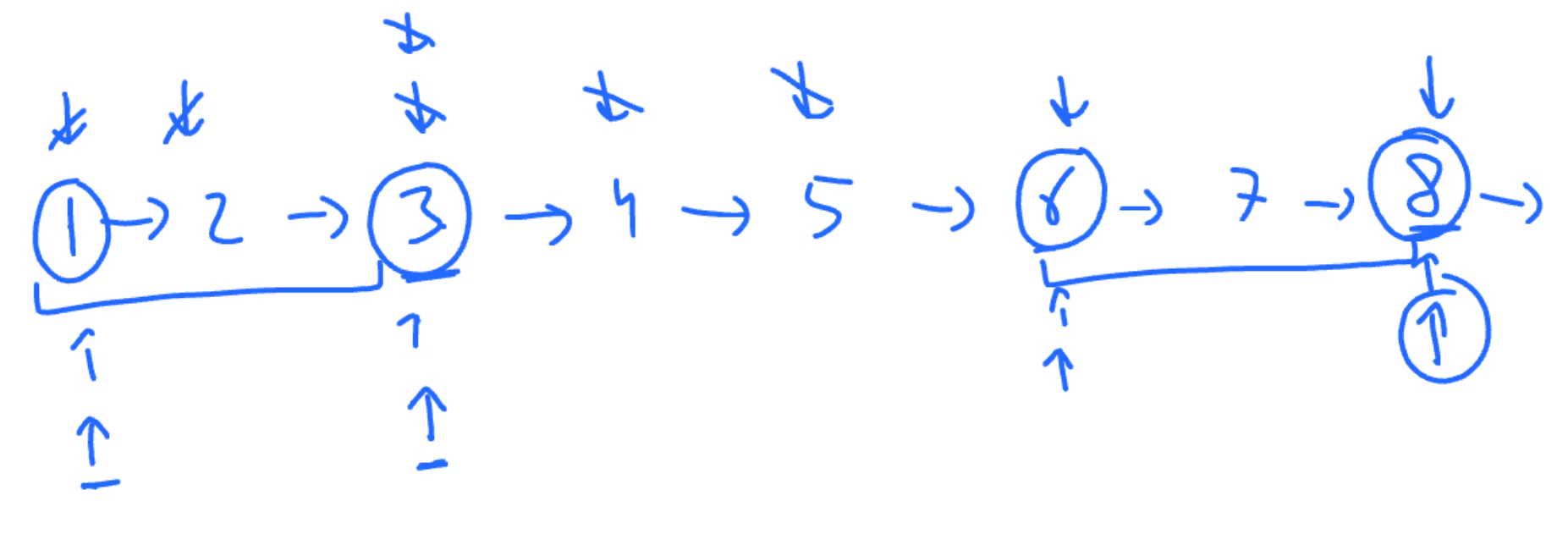
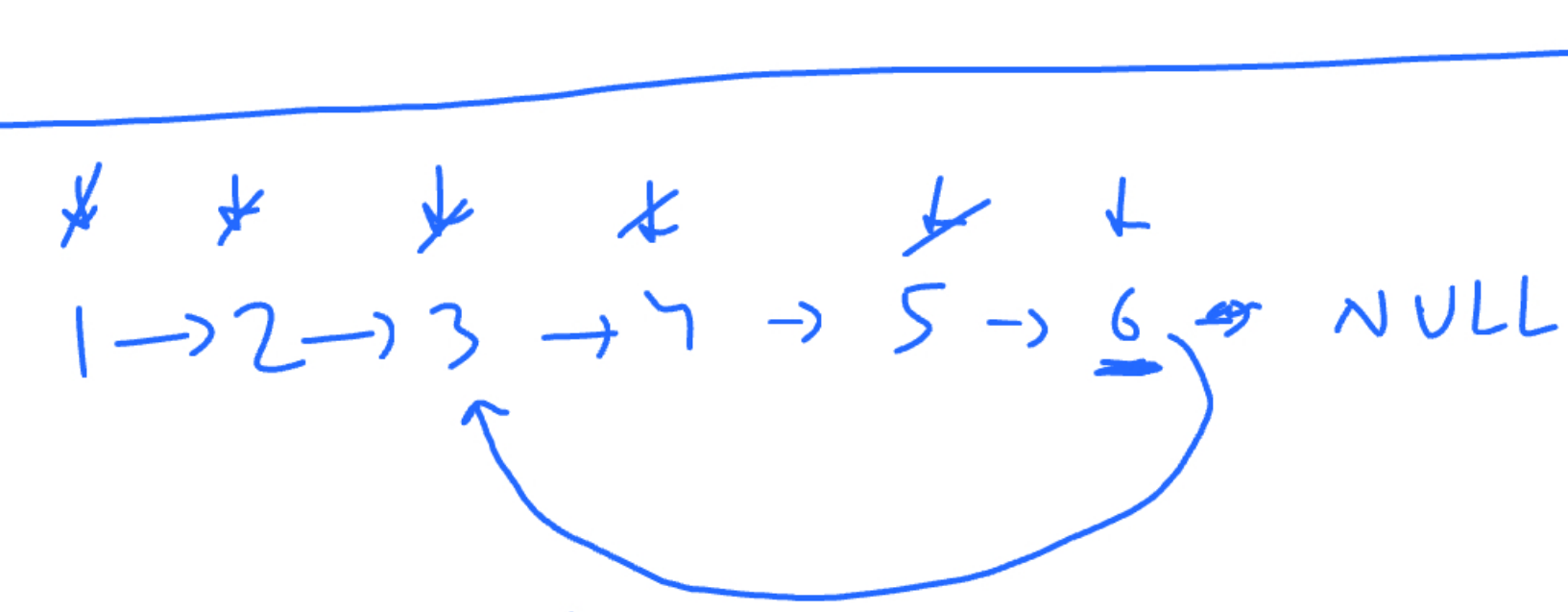


k=3.

→ n-k+1

8-3+1 = 6.



Visited.

2x

→ 3x

4x

5x

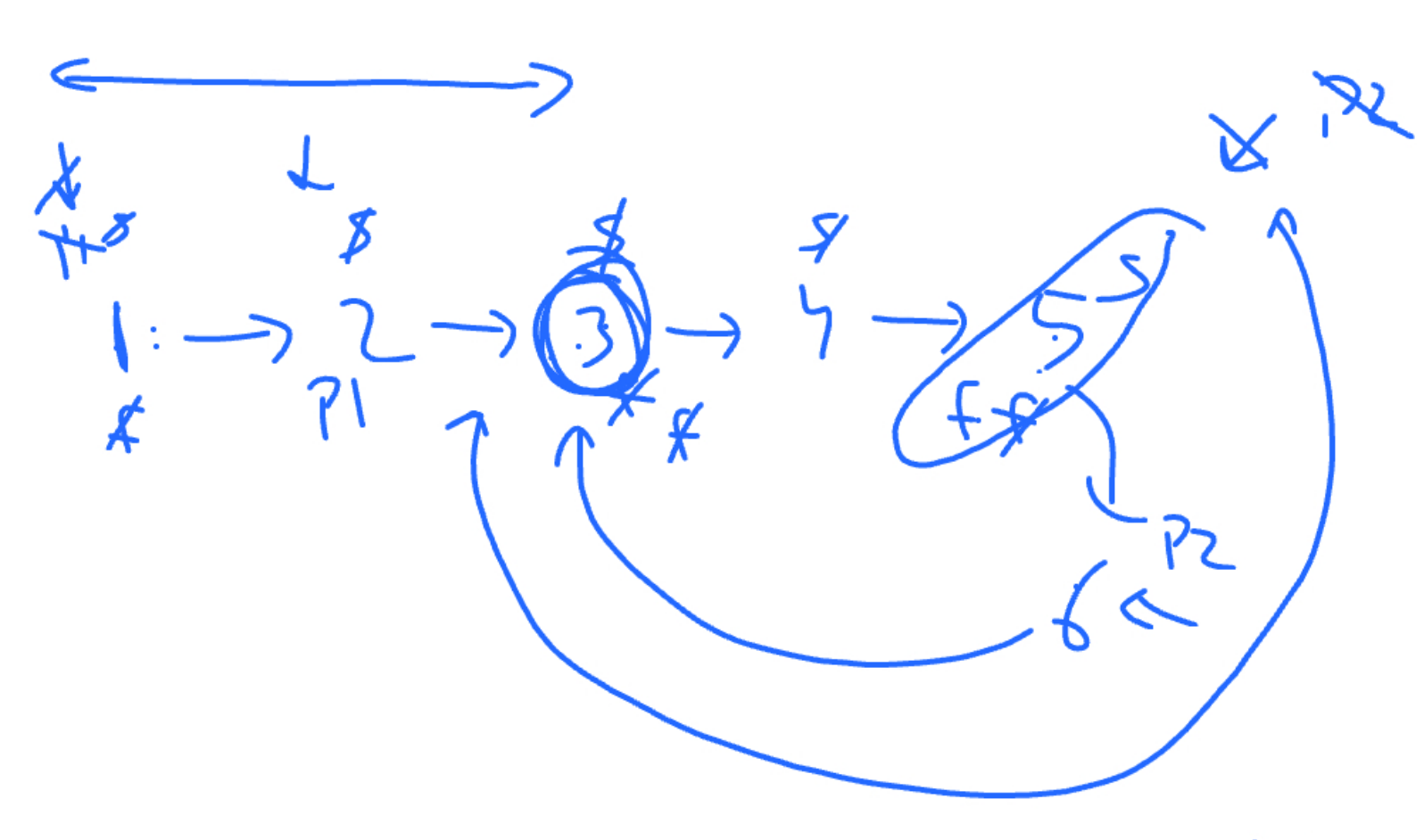
6x

Detection & Removal.

BF. - TC: O(n).

- AS: O(n).

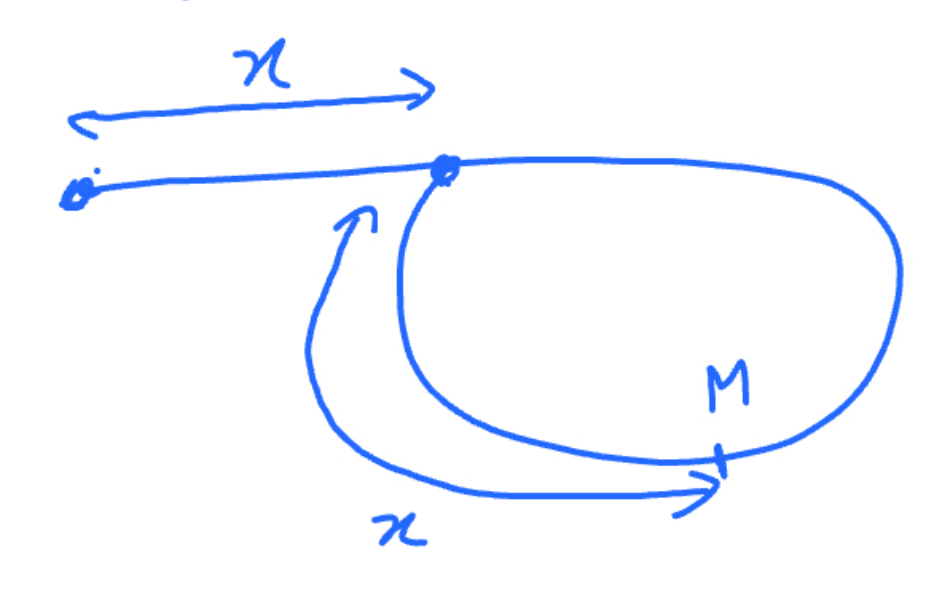
Optimal:

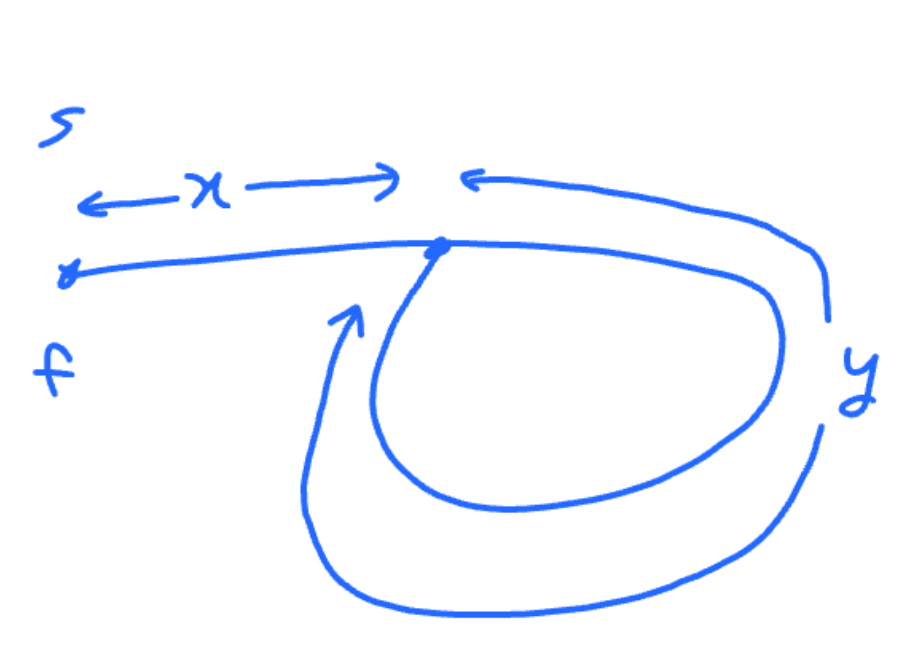


TC: O(n).

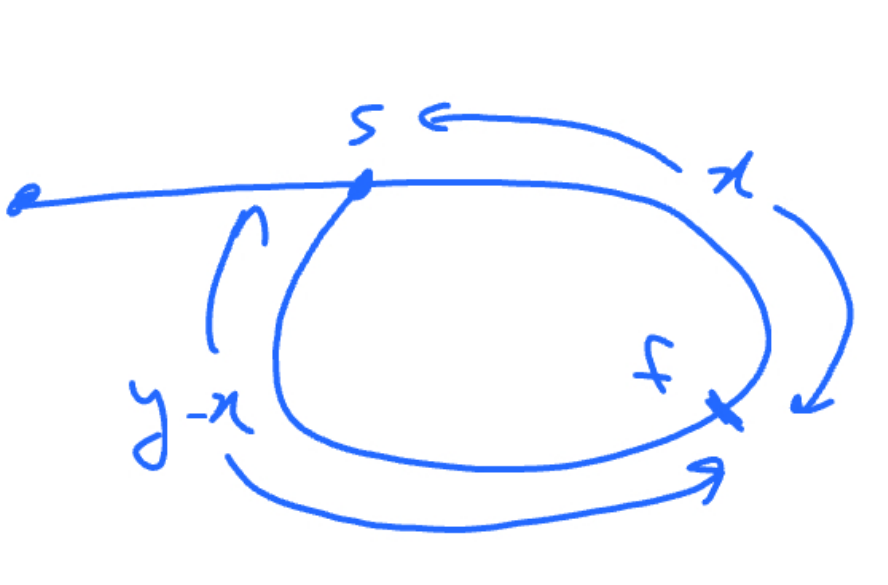
AS: O(1).

Floyd's Cycle Detection Algo.





At t = ~~0~~ - x



At t = ~~0~~ 0

slow is x steps behind fast

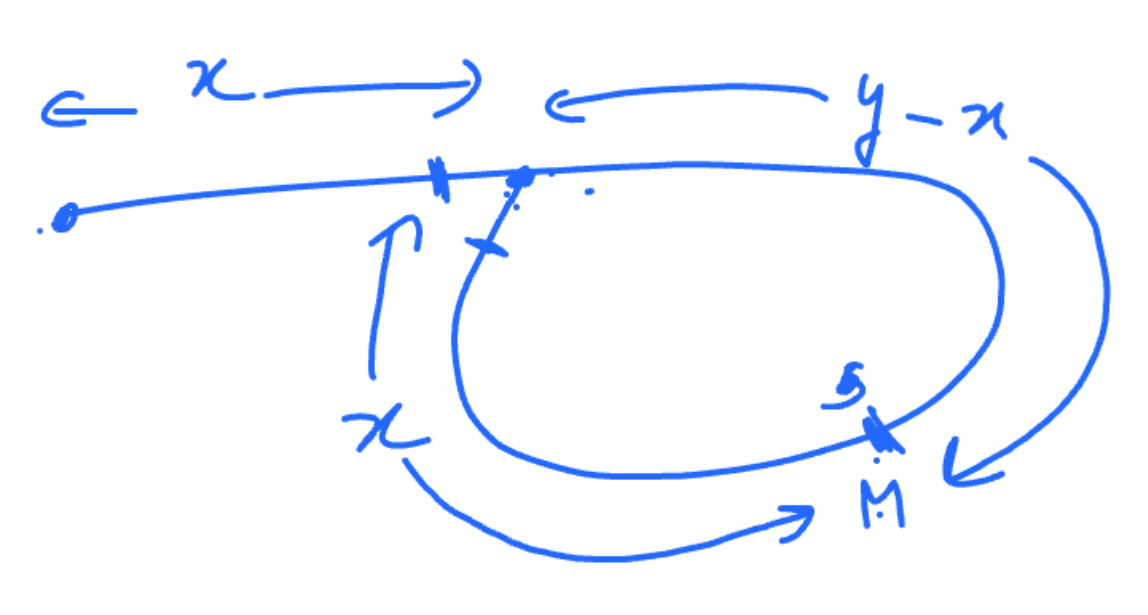
fast is (y-x) steps behind slow.

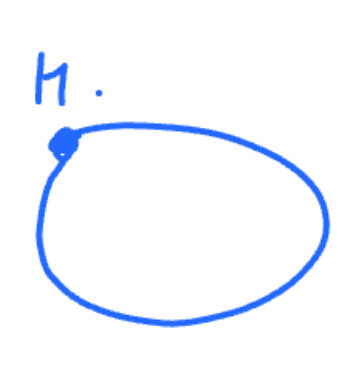
fast catches up slow 1 step per iteration.

Time taken by fast to catch up slow =

$t = \frac{d}{s}$

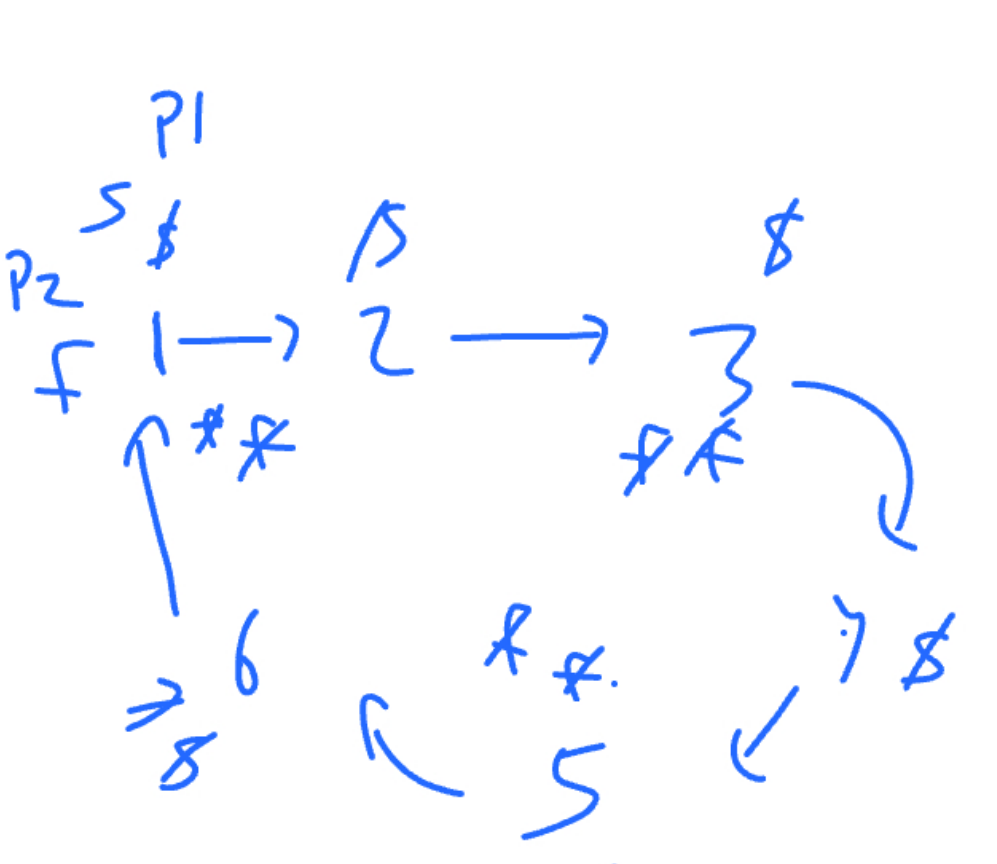
$t = \frac{(y-x)}{2-1} = y-x$

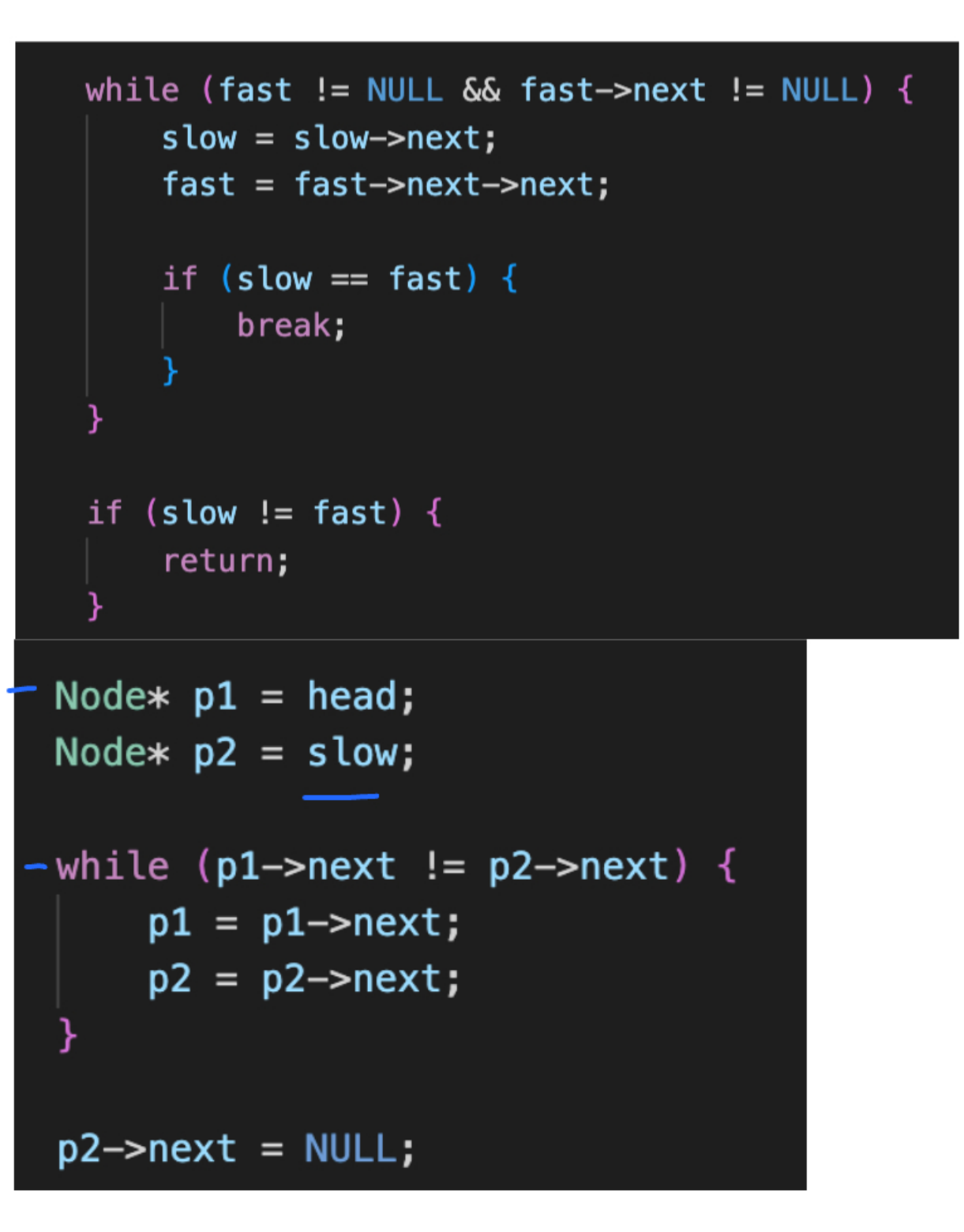




s°

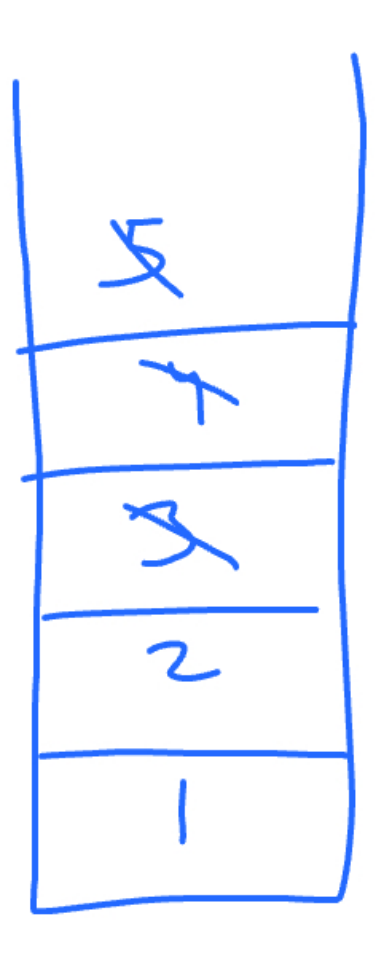
•f (100-50) = 50.





```
while (fast != NULL && fast->next != NULL) {
    slow = slow->next;
    fast = fast->next->next;

    if (slow == fast) {
        break;
    }
}

if (slow != fast) {
    return;
}
Node* p1 = head;
Node* p2 = slow;

while (p1->next != p2->next) {
    p1 = p1->next;
    p2 = p2->next;
}

p2->next = NULL;
```

LIFO: Last In First Out.



class Stack {
private:
→ vector<int> arr;
⋮
public:
push()
pop()
top()
size()
isEmpty()
⋮
}

~~5 4 3 2~~ 1

Stack using Arrays.

stack	Array.	
push()	push_back()	→ O(1)
pop()	pop_back()	→ O(1).
top()	arr[n-1]	→ O(1).

Stack Using L.L.

stack.	L.L	
push()	insert at begin	→ O(1).
pop()	delete at begin	→ O(1)
top()	return head→data	→ O(1)

~~5~~ → 4 → 3 → 2 → 1

([ )] → X

(( )) ✓

[({ })] → ✓

[ ] ( ) → ✓

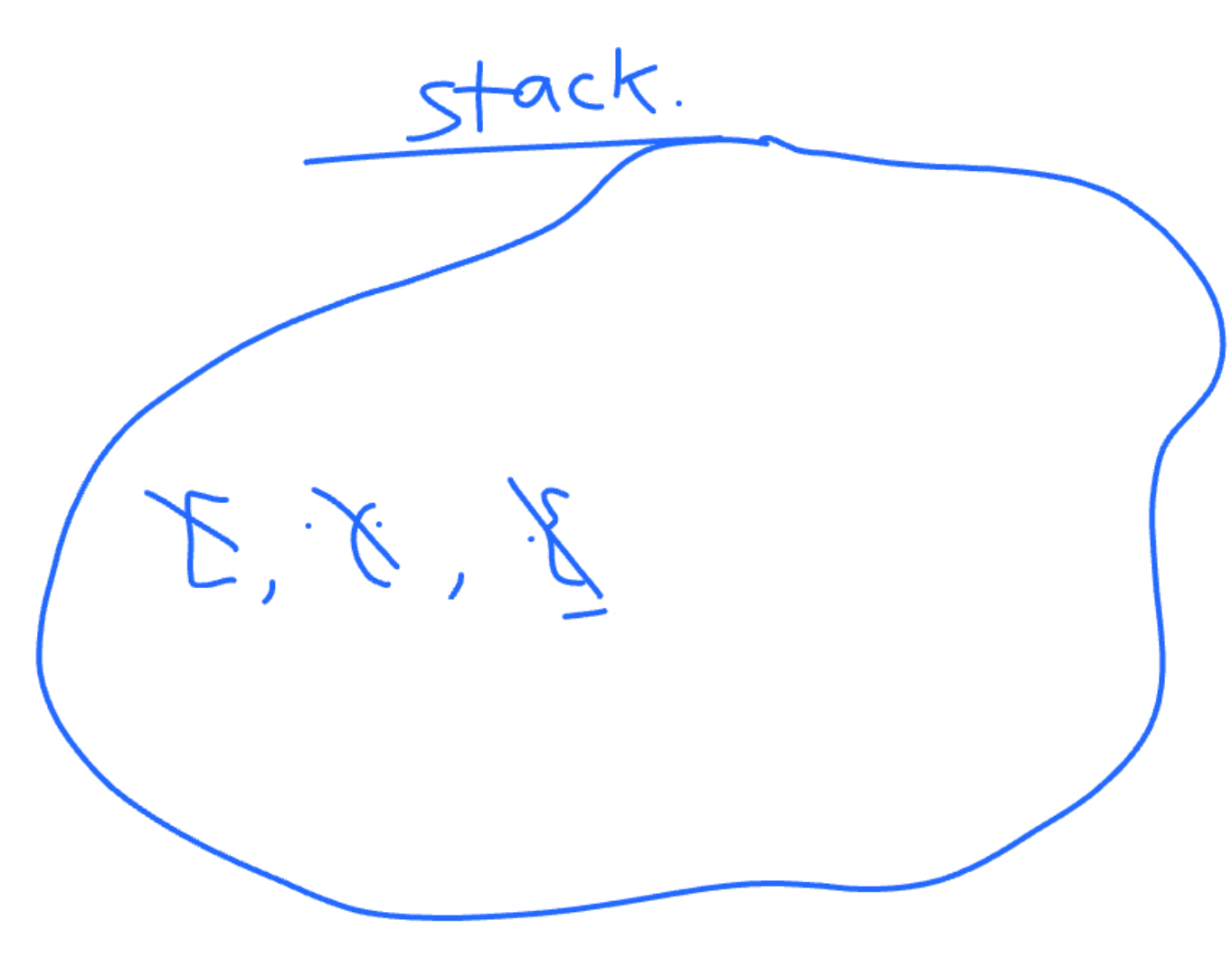
[({ })]

BF: TC: O(n²). AS: O(1).

))))))
0 1 2 3 4 5

[({ })]

stack.



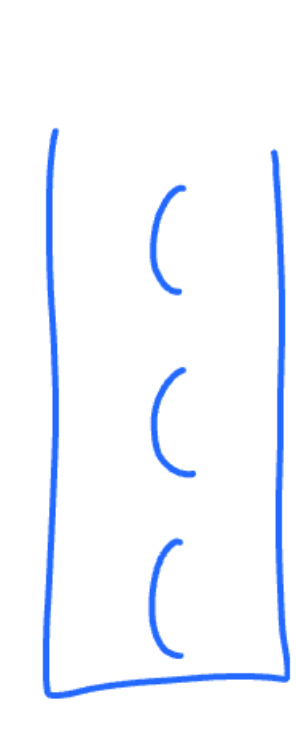
([ )]

X

( ) [ ]

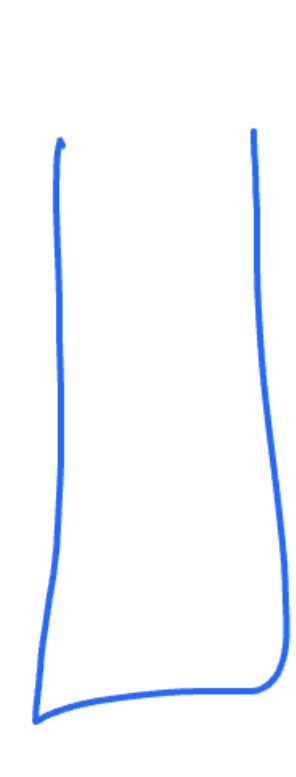
✓

(((



→ X

)))

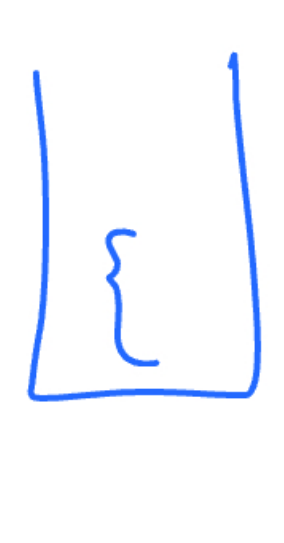


→ X

(|| { || [

TC: O(n).

AS: O(n)



}

[6, 8, 0, 1, 3]
8 -1 1 3 -1

[6, 8, 0, 3, 1]
8 -1 3 -1 -1

[6, 8, 0, 3, 1, 9, 10]
8 9 3 9 9 10 -1

BF: TC: O(n²) ← i: 0 → n-1
j: i+1 → n-1

AS: O(1)